

MIDTERM PRESENTATION

Course: Introduction to Artificial Intelligence

Code: 503043

Duration: 03 weeks

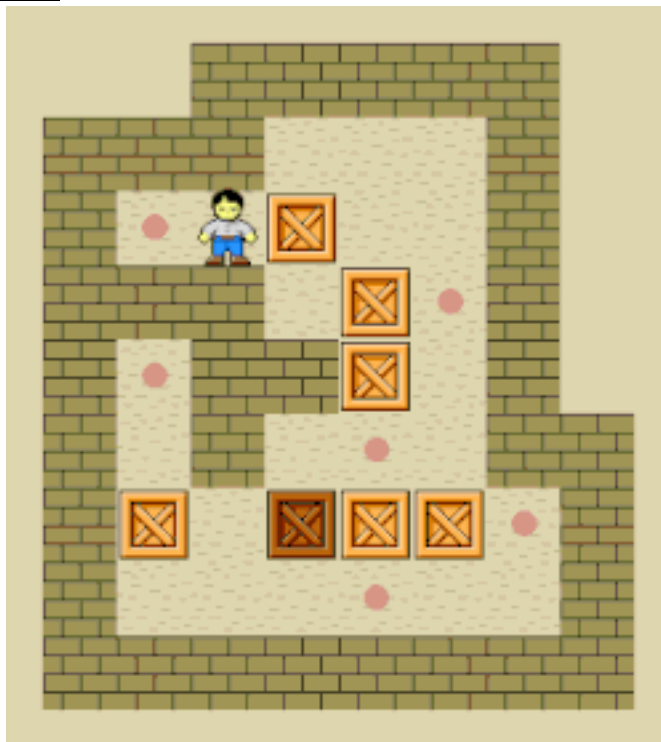
I. Formation

- The project is conducted in groups.
- Each group is required to complete the assigned tasks and submit the project according to the instructions below.

II. Tasks

a) Task 1 (8.0 points): Sokoban

Problem descriptions:



(Source: <https://en.wikipedia.org/wiki/Sokoban>)

- Students apply search strategies to enable the agent to move boxes to their designated positions.
 - Input: path to a layout file (*example_map.txt*)
 - Output: list of actions (North, East, West, South); total cost
 - The map structure is as below
 - % → obstacles/walls
 - A → initial location of the agent (the man)
 - B → boxes
 - D → designated positions of boxes (red points)

- C → a box is currently at a designated point (dark brown boxes)
- spaces → blank cells.

example_map.txt

```

%%%%%
%%%  %
%DAB  %
%%% BD%
%D%%B %
% % D %%
%B CBB D%
%   D   %
%%%%%%%%%

```

Requirements:

1. (1.0 point) Formulate the given problem as a state-space search problem and determine the details of its relevant components.
2. (1.0 point) Implement UCS and A* algorithms to solve the problem. Propose a heuristic function for A* algorithm. Note that Euclidean and Manhattan distances are not allowed.
3. (1.0 point) Propose and implement an experimental approach to contrast the time and the space complexity of the two algorithms.
4. (1.0 point) Discuss the properties of the proposed heuristic function, including its admissibility and consistency. Design and implement an experiment to verify these two properties, and report the experimental results.
5. (1.0 point) Use the **pygame** library to implement the game's GUI. Ensure that the game is user-friendly. The UI and UX will be evaluated based on the lecturer's overall assessment.
 - There are two options of algorithms, including UCS and A*.
 - Display the number of actions on the UI.
 - The user can pause the game, move forward, and backward using Space, →, and ← keys respectively.
 - Organize the program following the object-oriented programming (OOP) model. Ensure that the source code is compact, well-structured, and reasonable.
 - Ensure that the project can be executed on macOS 13.7.8 (Ventura) with an Intel Core i5 processor. Carefully verify the versions of all relevant Python libraries.
6. (1.0 point) Reformulate the problem as a competitive two-agent problem.

- The two agents compete to move boxes to their designated positions. After a given number of steps n , the agent that completes more boxes is declared the winner. Number n is input by the user.
 - An agent can move a box that has already been placed in its designated position by the other agent out of that position and then place the box again.
 - At each game step, the two agents must choose and perform their actions simultaneously.
 - The two agents cannot pass through each other.
 - You may design a larger map to provide a suitable environment for the two agents to compete.
7. (1.0 point) Design an algorithm to control the agents based on one or more of the algorithms covered in the lectures, such as BFS, UCS, DFS, DLS, IDS, GBFS, or A.
- The time limit for each decision-making step is 1,000 ms.
8. (1.0 point) Reimplement the game described in Requirement 5 so that it supports the competitive gameplay.
- Boxes occupied by different agents should be displayed in different colors.
 - The algorithms controlling the two agents must be implemented in separate source code files, allowing student groups to compete against each other.

Notices:

- Recommended editor: Visual Studio Code
- b) Task 2 (2.0 points): Presentation**
- Each student group must prepare a presentation to report on their work using their own template.
- The presentation must include the following content:
 - Student list: Student ID, full name, email address, assigned tasks, and completion percentage.
 - Brief presentation of the approaches used to solve the tasks, making use of pseudocode and/or diagrams where appropriate.
 - Avoid embedding raw source code in the presentation.
 - Advantages and disadvantages of the proposed approaches.
 - A table showing the completion percentage for each task.
- Format requirements:
 - Use a 4:3 slide aspect ratio. Avoid dark backgrounds and colorful shapes due to projector limitations.

- Students must ensure that all content is sufficiently clear and readable when the presentation is printed in grayscale.
- Presentation duration: The presentation must not exceed **05 minutes**.

III. Submission Instructions

- Create a folder whose name is as
AI_midterm_<project group ID>_<your student ID>
- Content:
 - **source** → project folder, each task is located in a subfolder
 - **presentation.pdf** → presentation.
 - **demo.txt** → URL to the demo video with **the maximal duration of 03 minutes**.
- Compress the folder into a zip file and submit by the deadline.
- All member must submit the project.

IV. Policy

- **Student groups that submit their projects late will receive 0.0 points for each group member.**
- **Missing any required materials from the submission will result in a deduction of at least 50% of the presentation score.**
- **Copying source code from the Internet or other students, sharing your work with other groups, or engaging in similar forms of academic misconduct will result in a score of 0.0 for all groups involved.**
- **If there are any indications of unauthorized copying or sharing of the assignment, additional interviews may be conducted to verify the authenticity of the students' work.**
- **The use of AI tools is prohibited in this project. If any AI-generated symbols or other identifiable AI-generated content are found in the source code files, the group will receive a score of 0.0 for the corresponding task.**

-- THE END --